

## Scoring System Thoughts - Hermann

### 1. Core System Layers

- a. Eligibility; Can this song even be in the live queue?
  - i. Handles:
    - 1. venue blocked artists / genres / explicit rules
    - 2. duplicate song lockout
    - 3. max active requests per user
    - 4. cooldowns
    - 5. session validity
    - 6. location / venue presence later
    - 7. minimum quality gates later
  - ii. Binary; eligible = true / false
- b. Ranking: Among eligible songs, what order should they be in?
  - i. Queue\_rank\_score
- c. Playability/Triggering: Does the top-ranked song actually play now?
  - i. This is where later we can add:
    - 1. minimum active users
    - 2. minimum positive ratio
    - 3. "super boost requires approval threshold"
    - 4. suggestion mode requiring venue approval
    - 5. venue anthem activation logic
    - 6. sponsor-song unlock logic
  - ii. This is separate from ranking. A song can be ranked #1 but still fail a playability rule.
- d. Overrides: Temporary things that can supersede normal ranking.
  - i. For example:
    - 1. venue plays house track manually
    - 2. Pass the Aux winner gets 1 guaranteed next song
    - 3. sponsor activation
    - 4. venue anthem
    - 5. bartender promo code event
    - 6. future brand promotions
  - ii. These should live above the normal queue logic as controlled override events.
- e. Settlement: What was paid for, what was earned, what gets refunded, what becomes account credit? Separate from ranking. This includes:
  - i. coin purchases
  - ii. per-action charges
  - iii. refunds if paid song never plays
  - iv. venue/company rev share
  - v. promo code redemption
  - vi. prepaid credit accounting
- f. Analytics: Everything should be logged independently of live queue logic.
  - i. request created

- ii. vote added
- iii. vote removed
- iv. boost purchased
- v. song played
- vi. song skipped
- vii. user session started
- viii. user session expired
- ix. venue override used

## 2. Score Architecture

- a. Every eligible song gets a score. Highest score ranks first.
- b.  $\text{QueueRankScore} = \text{DemandScore} + \text{PaymentScore} + \text{ReputationScore} + \text{ContextScore} - \text{FrictionScore}$ 
  - i. DemandScore - crowd demand
    - 1. original request
    - 2. upvotes
    - 3. downvotes
    - 4. maybe unique requesters later
    - 5. vote velocity later
  - ii. PaymentScore - paid influence
    - 1. Paid boosts
    - 2. drink promo code credits
    - 3. sponsor boosts
  - iii. ReputationScore - Who is driving the demand?
    - 1. user influence
    - 2. trusted users
    - 3. anti-spam penalties
    - 4. group-abuse penalties
  - iv. ContextScore - Venue/night context.
    - 1. genre boost during Power Hour
    - 2. venue preference
    - 3. time-of-night fit
    - 4. future mood / continuity logic
    - 5. venue anthem / sponsor logic if you want that folded in
  - v. FrictionScore - Things pushing songs down without banning them.
    - 1. age decay
    - 2. repeated artist fatigue
    - 3. recent duplicate artist penalty
    - 4. spam suspicion penalty

## 3. V0 Base Data Model

- a. Each live queue item should have at least:
  - i. queue\_item\_id
  - ii. venue\_id
  - iii. song\_id
  - iv. requesting\_user\_id

- v. created\_at
- vi. status (queued / playing / played / skipped / expired / blocked)
- vii. upvotes\_count
- viii. downvotes\_count
- ix. unique\_supporter\_count
- x. priority\_boost\_count
- xi. instant\_vote\_count
- xii. super\_boost\_count
- xiii. explicit\_flag
- xiv. genre
- xv. artist
- xvi. Is\_duplicate\_locked

b. each user-venue session should have:

- i. session\_id
- ii. user\_id
- iii. venue\_id
- iv. joined\_at
- v. last\_active\_at
- vi. is\_guest
- vii. active\_request\_count
- viii. Cooldown\_ends\_at

c. Each payment event:

- i. payment\_event\_id
- ii. user\_id
- iii. venue\_id
- iv. queue\_item\_id (if applicable)
- v. payment\_type
- vi. credit\_amount
- vii. cash\_amount
- viii. created\_at
- ix. status
- x. Refund\_status

4. V0 Score Formula

a.  $\text{QueueRankScore} = \text{DemandScore} + \text{PaymentScore} - \text{FrictionScore}$

- i.  $\text{DemandScore} = \text{RequestBase} + \text{UpvoteWeight} * \text{upvotes\_count} - \text{DownvoteWeight} * \text{downvotes\_count} + \text{UniqueSupporterWeight} * \text{unique\_supporter\_count}$
- ii.  $\text{PaymentScore} = \text{PriorityBoostWeight} * \text{priority\_boost\_count}$
- iii.  $\text{FrictionScore} = \text{ArtistRepeatPenalty} + \text{SpamPenalty}$

b. V0 default weights

- i. RequestBase = 2 ; A newly requested song should not sit at zero strength. A request is already meaningful intent.
- ii. UpvoteWeight = 1; Simple baseline unit.

- iii. DownvoteWeight = 1.25; Downvotes should matter slightly more than upvotes, because they signal active resistance, not just passive lack of interest.
    - iv. UniqueSupporterWeight = 0.5; A song getting support from distinct people is valuable, but you do not want to double count too aggressively yet.
    - v. PriorityBoostWeight = 3; This means a \$1 boost is meaningful but not absolute. It is roughly equal to a few strong votes, not an override.
      - 1. priority\_boost\_count += 1 for every boost
  - c. Score tiebreakers; by order
    - i. Higher unique supporter count
    - ii. Earlier created at - First in wins ties.
    - iii. Lower downvote counts - Less resisted song wins.
  - d. Additional v0 data model objects
    - i. Queue
      - 1. last\_score\_calculated\_at
      - 2. current\_score
      - 3. playability\_state
      - 4. playability\_reason
      - 5. source\_type (organic, sponsor, venue, override)
    - ii. Sessions
      - 1. is\_active
      - 2. session\_expired\_at
      - 3. last\_vote\_at
      - 4. Last\_request\_at
    - iii. Analytics
      - 1. event\_id
      - 2. event\_type
      - 3. event\_timestamp
      - 4. actor\_user\_id
      - 5. venue\_id
      - 6. queue\_item\_id
      - 7. metadata\_json

#### Estimation Rating System- Alex

1-3 = Easy (Done within a day)

5 - 8 = Medium(few days to a week)

13 - 21 = Hard(Multiple weeks)

Implemented(already done)

Clarify(Specify or Explain Further)

#### Feature Prioritization - Hermann

- V0
- Alex- Adding Sign in feature for users to v0

Should allow users to sign in to have their settings and stats saved and to be able to buy coins/tokens in bulk, take advantage of discounts, etc. Guest users should be limited to immediate use purchases only -5

- QR Venue Join -Implemented
- Song Request System -Implemented
  - Crowd voting queue -Implemented
    - How much paid-boosts -Clarify
    - When it was requested -Implemented
    - Upvotes/downvotes -Implemented
- Anti-spam
  - QR join + session expires if user inactive or a certain amount of time (1 hour) -1
  - Max 3 active requests by user -Implemented
  - Cooldown: 1 request every 2 minutes -1
  - Same song cannot be requested more than once every 45 minutes -Implemented
- Monetization
  - Function to accept payment from users, pay out to venues/our company -5
  - Prepaid credits
    - Tracking system to keep account -5
    - Priority Boost \$1 - Moves your requested song up the queue by a few spots (ex: +3 positions). If the song already has votes, it jumps ahead of other similar-scoring songs. Limit to 1 per song per user to prevent spam. -2
    - Refund mechanism in case you paid for premium options that don't get played, still influence your user score; weight of unique song should play into negative impact -5
  - Venue's Anthem -5
    - Bars can attach songs to drink specials.
      - Tonight's Sponsor Song
      - Not Like Us – Kendrick Lamar
      - Play this song:
      - \$2 off tequila shots or 20% off etc.
        - Song gets requested
        - If it wins → promotion activates for 5 minutes
        - Everyone rushes the bar
  - Monetization-Friendly UI
    - Users spend money when:
      - Their song is close to playing
      - Someone else just boosted
      - They want to beat another user
    - So your UI should show:
      - Your song: #6 in queue -1

- Boost to #3 for \$1-2
  - Venue Control
    - Option to begin with a venue loaded playlist/queue?-3
    - Control Modes
      - Crowd Control - fully controlled by crowd but overridable at any time by venue-1
        - Venues can block - lets consider this carefully, does it defeat whole point if supply house can just keep to its usual-3
          - explicit songs
          - genres
          - artists
          - tempo ranges, maybe after a certain time etc  
-Clarify(Not as easy as it sounds i.e Hot by Gunna and Sing About Me Kendrick have the same bpm)
      - Suggestion mode - venue approves-1
  - Announcements-Implemented
    - Next song-Clarify(Does this need to be “announced” if the now playing and up next queue are already visible )
    - Name of user who ‘won’ current song; “DJ Alex is playing...” or something similar-1
    - Venue anthem announcements-1
    - Estimated play time till song plays based on position in queue; how many minutes shaved off if boost used
  - Analytics tracking (simply logging the data, no need for display or other interaction yet)
    - Songs requested through OpenAux vs all songs played at venue-1
    - Revenue generated-1
    - Votes per song-1
    - Active users per venue-1
    - Track by geography/time/venue type:-3
      - Most requested songs
      - genre
      - real-time club trends
      - skip rates
- V1
  - Song Request System
    - Crowd voting queue-Implemented
      - Display of 2 lists, what is supposed to be next 3, and larger randomized list of all songs-2
    - Reputation-Based Weighting - users get influence scores so more respected users have increased voting power. For example:
      - upvotes from others + -1

- spam attempts – -2
  - songs skipped (by venue, any other situation song would be skipped?) – Song could be skipped by a crowd voted skipping mechanism? -1
  - Downvotes? -1
- Anti-spam-8
  - Location verification within reasonable radius of venue-5
  - Minimum threshold of votes on a song to play? Maybe threshold is activated only when there are a minimum number of ‘active’ users  
-Clarify(Should this mean no songs play if not enough votes? Seems too harsh)
    - If certain amount of people want to hear same song, then override (70% of active users)?
    - if same song is requested by multiple people (not within the same ‘group’, greater weight than an upvote, 1.5x?
  - Prevent group abuse (possibly move to later)
    - Detect group:-8
      - Same arrival time -2
      - Same table/location (WiFi / Bluetooth proximity) or micro radius, over time -8
      - Same social graph (friends)-5
      - Same song voting patterns, consistent over time-5
- Monetization
  - Prepaid credits
    - Instant Play Vote \$3 - Adds a large number of votes instantly to a song (ex: equivalent to 10 votes). Helps a song reach the threshold to play sooner. Cannot exceed max vote weight so the crowd can still override (for example, Song Score = Crowd votes × 1 + Paid votes × 0.4 + User reputation × 0.3 ) -3(We should reach consensus)
  - Power Hour Mode - drink totals for genre based time control (venues manually activate through their side of the app)
    - 4 tequila shots - Genre boost (Hip Hop x2 votes for 15 minutes)
    - Display:
      - FE!N – Travis Scott
      - 🔥 Boosted by 4 Tequila Shots
  - Buy a drink, boost a song
    - Or bartender gives a Boost Code (venues can generate promo codes in their side of app for qualifying purchase); promo code expires after certain time. 30 min?-Clarify(We need to discuss implementation)
    - For example:
      - Beer +1 boost credit
      - Cocktail +2 boost credits

- Bottle service +10 credits (super boost with less of minimum threshold of upvotes)
    - Dispute system in case of venue abuse 13(Needs Discussion)
  - Monetization-Friendly UI
    - Users spend money when:
      - Their song is close to playing
      - Someone else just boosted
      - They want to beat another user
    - So your UI should show:
      - Instant Vote for \$3 Clarify(Instant Vote or Instant Play?)
- V2
  - Song Request System
    - Reputation-Based Weighting - users get influence scores so more respected users have increased voting power. For example:
      - songs that got played +
      - time spent in venue +
      - You should get rewarded for any engagement
  - Anti-spam
    - Max 5 active requests per 'premium' user
      - How is premium defined? Either purchase of paid options that night at that venue, or a minimum user influence score; think the former leads to better monetization
  - Monetization
    - Prepaid credits
      - Super Boost \$5 - Temporarily forces the song to the front of the queue if it passes minimum approval (ex: 30% upvotes). Think of it as a paid "skip the line." Require a minimum positive vote ratio so people can't force terrible songs.
    - Monetization-Friendly UI
      - Users spend money when:
        - Their song is close to playing
        - Someone else just boosted
        - They want to beat another user
      - So your UI should show:
        - Super Boost for \$5
  - Venue Control
    - Dashboards
      - Crowd Mood Detection using signals like
        - song skip rate
        - vote velocity
        - time of night
        - genre engagement
        - To show venue:



- Crowd energy: HIGH
    - Trending: Afrobeat / hip hop
    - Low engagement: house
  - V3
    - Song Request System
      - Reputation-Based Weighting - users get influence scores so more respected users have increased voting power. For example:
        - crowd reactions (see further below)
        - Unique songs
    - Monetization
      - “Beat the Song Above You” Pricing. This creates direct competition. Instead of a fixed boost, the UI shows:
        - Your Song: #4
        - Beat #3 for \$1.20
        - Beat #2 for \$2.80
    - Social/Engagement
      - Vibe Score
        - Real time reactions displayed on app (floating upon reaction and summary as seen further below) and venue if display available
          - 👍 Fire
          - 💃 Danceable
          - 😊 Mid
          - 🙅 Skip
            - NOW PLAYING
            - Drake – Passionfruit
- 🔥 62%
    - 💃 25%
    - 😊 8%
    - 🙅 5%
- Analytics tracking (UI interface for the below with target audience being record labels, DJs, etc)
  - Track by geography/time/venue type:
    - Most requested songs
    - genre
    - real-time club trends
    - skip rates
- V4
  - Monetization
    - Dynamic Pricing of credits based on competition; marked by some combo of # of active users and level of engagement at the venue. Set price caps maybe.
    - Brand Promotions

- Play this song sponsored by Red Bull. Free shot if it reaches 100 votes.
  - Leaderboard - Top DJs Tonight - rewards/discounts for leaders?
    - Alex – 9 songs played
    - Maya – 7 songs played
    - Sam – 6 songs played
      - Can be venue specific, geography specific, and time differentiated (tonight, weekly, all-time)
    - Way to have people at different venues compete against each other?
      - Maybe for brand sponsored rewards?
  - Pass the Aux
    - Every [20] minutes the system selects a temporary crowd DJ who gets to queue up to [1] guaranteed songs to played next
      - Rule for who gets or random?
        - Random of people who've built greatest user influence scores for night
      - Builds user influence scores
      - Display: 🔥 Alex has the AUX for 10 minutes
- V5
  - 
  - Live Venue Dashboard
    - Where music is good tonight? (think persistent leaderboards can lead to crowding out of smaller/lesser known venues)
      - What metrics to use for this
        - Level of positive engagement (weight of pure activity vs positive scores)
    - Where this genre has been popular tonight
    - Try this tonight (in support of lesser known venues)
      - Maybe increase positive effect on user influence scores/decrease negative while at this venue
      - Some moderated, maybe some sponsored
  - Social Profiles (not data side but minimal engagement between users)? Users can choose to have a 'public' profile or not
    - See genre tastes, most requested songs/artists
    - Keep in mind, don't want to open up shaming based on influence scores (if we even decide to publicize those in some way)
    - Analytics tracking tied to demographics associated with user profile (UI interface for the below with target audience being record labels, DJs, etc)
      - Track by geography/time/venue type:
        - Most requested songs
        - genre
        - real-time club trends
        - skip rates

- V6
  - AI Auto Queue
    - The algorithm decides songs so song continuity makes sense. For example, song scores based on:
      - votes
      - tempo
      - genre continuity
      - Key compatibility
      - crowd reactions
      - time of night
      - venue preferences/venue audience/venue location
  - Monetization
    - Ads

General Brand identity Concepts/Taglines

<https://arlyne-endurant-laurine.ngrok-free.dev/>

**This Link only works if i have the program actively running on my pc so if its offline and you want to test, lmk and i'll spin it up**

**Test Links:**

- Music to the people.
- Not just “control the music”, “find the best music in the city tonight.”
- discover bars/venues through live crowd music.

Sensitivities

- Location requests

Product Functionality

A. General

a. Song Request System

- i. Crowd voting queue - queue automatically sorts based on number of requests (and upvotes/downvotes) for any particular song
  1. 2 lists, what is supposed to be next 3, and larger randomized list of all songs
  2. How do we consider position in queue
    - a. How much paid-boosts
    - b. When it was requested
    - c. Upvotes/downvotes

- ii. Reputation-Based Weighting - users get influence scores so more respected users have increased voting power. For example:
  1. songs that got played +
  2. songs skipped -
  3. upvotes from others +
  4. time spent in venue +
  5. spam attempts -
  6. crowd reactions (see further below)
  7. Unique songs
  8. You should get rewarded for any engagement
  - 9.
- iii. Anti-spam limits
  1. Max 3 active requests per user
    - a. Make premium have more (maybe 5?)
  2. Cooldown: 1 request every 2 minutes
  3. Same song cannot be requested more than once every 45 minutes
  4. Minimum threshold of votes on a song to play? Maybe threshold is activated only when there are a minimum number of 'active' users
    - a. If certain amount of people want to hear same song, then override (70%)?
    - b. if same song is requested by multiple people, greater weight than an upvote,
      - i. Let's think about a way to make sure groups don't game system, track mutual friends through instagram?
  5. If you spam downvotes/upvotes you have less weight or are restricted
  6. You should r
- b. Premium/Paid (Think about revenue split for app (30%) vs venue (70%))
  - i. Prepaid credits
    1. Priority Boost \$1 - Moves your requested song up the queue by a few spots (ex: +3 positions). If the song already has votes, it jumps ahead of other similar-scoring songs. Limit to 1 per song per user to prevent spam.
    2. Instant Play Vote \$3 - Adds a large number of votes instantly to a song (ex: equivalent to 10 votes). Helps a song reach the threshold to play sooner. Cannot exceed max vote weight so the crowd can still override (for example,  $\text{Song Score} = \text{Crowd votes} \times 1 + \text{Paid votes} \times 0.4 + \text{User reputation} \times 0.3$ )
    3. Super Boost \$5 - Temporarily forces the song to the front of the queue if it passes minimum approval (ex: 30% upvotes). Think of it as a paid "skip the line." Require a minimum positive vote ratio so people can't force terrible songs.

4. Refund mechanism in case you paid for premium options that don't get played, still influence your user score; weight of unique song should play into negative impact
- ii. Venue Product Tie-In (Display with OpenAux Specials)
  1. Buy a drink, boost a song
    - a. Receipt QR code
    - b. Or bartender gives a Boost Code
    - c. For example: do we just let venues freely choose the below? Feel like credit to cost has to be regulated
      - i. Beer +1 boost credit
      - ii. Cocktail +2 boost credits
      - iii. Bottle service +10 credits (super boost with less of minimum threshold of upvotes)
  2. Venue's Anthem
    - a. Bars can attach songs to drink specials.
      - i. Tonight's Sponsor Song
      - ii. Not Like Us – Kendrick Lamar
      - iii. Play this song:
      - iv. \$2 off tequila shots
        1. Song gets requested
        2. If it wins → promotion activates for 5 minutes
        3. Everyone rushes the bar
  3. Power Hour Mode - drink totals for genre based time control
    - a. 4 tequila shots - Genre boost (Hip Hop x2 votes for 15 minutes)
    - b. Display:
      - i. FE!N – Travis Scott
      - ii. 🔥 Boosted by 4 Tequila Shots
- iii. Monetization-Friendly UI
  1. Users spend money when:
    - a. Their song is close to playing
    - b. Someone else just boosted
    - c. They want to beat another user
  2. So your UI should show:
    - a. Your song: #6 in queue
    - b. Boost to #3 for \$1
    - c. Instant Vote for \$3
    - d. Super Boost for \$5
- c. Possible Algorithm Logic

Here's a voting algorithm that's simple to build, hard to abuse, and still feels "fair" in a loud bar.

## Core idea

Don't decide songs by raw votes. Decide by a **Song Score** that blends:

- **crowd support**
  - **paid influence (capped)**
  - **anti-spam + anti-troll**
  - **time/queue fairness**
  - **venue vibe constraints**
- 

## 1) Data you track per song request

For each requested song  $s$ :

- $up(s)$  = unique upvotes (people in venue)
  - $down(s)$  = unique downvotes
  - $paid(s)$  = paid credits spent on this song (Priority/Instant/Super)
  - $age(s)$  = minutes since request
  - $rep(u)$  = user reputation (optional but powerful)
  - $dupe\_penalty(s)$  = if same user/crew spams variants/remixes
- 

## 2) Convert paid actions into “paid points” (not raw votes)

Treat purchases as points, then **soft-cap them**.

Example mapping:

- Priority Boost (\$1) = +1 paid point
- Instant Play Vote (\$3) = +4 paid points
- Super Boost (\$5) = +7 paid points

Now apply a cap:

- `paid_points_capped(s) = min(paid_points(s), cap)`
- cap could be **25–40% of what's needed to win**

So no one can “buy” #1 outright.

---

### 3) Simple score formula (works well)

#### Song Score

$$\text{score}(s) = A \cdot \text{net\_votes}(s) + B \cdot \text{paid\_points\_capped}(s) + C \cdot \text{time\_boost}(s) - D \cdot \text{skip\_risk}(s) - E \cdot \text{spam}(s)$$

Use:

- `net_votes = up - 0.7*down` (downvotes matter, but not too much)
- `time_boost = log(1 + age)` (older requests slowly rise)
- `skip_risk` grows if downvote ratio is high
- `spam` triggers on repeat requests, bots, rapid-fire voting

Suggested weights to start:

- `A = 1.0`

- $B = 0.6$  (paid helps, but weaker than real crowd)
- $C = 0.4$
- $D = 2.0$
- $E = 3.0$

(You'll tune these per venue.)

---

## 4) Hard guardrails that prevent most abuse

### A) Minimum crowd approval to be eligible

A song can only play if:

- $up \geq 6$  and
- $up / (up + down) \geq 0.60$

So even with paid points, a song can't play if the room hates it.

### B) Per-user influence limit

Per user per 10 minutes:

- max **3 votes cast**
- max **1 paid action**
- max **1 request**

### C) Crew / collusion detection (optional but huge)

If 6 people who arrived together are voting the same way instantly:

- reduce their marginal vote weight (soft).



---

## 5) Queue selection rule (the “DJ brain”)

Instead of always picking the top score, do:

1. Build **eligible set** (passes approval + not blacklisted + fits venue rules)
2. Pick the highest score **that also fits vibe constraints**, like:
  - avoid same artist within last X songs
  - keep BPM within  $\pm 15$  of current track unless late night
  - respect explicit venue “do-not-play” list

This keeps transitions smooth and venues happy.

---

## 6) What each monetized feature does inside this model

- **Priority Boost (\$1)**: +1 paid point AND +small “age jump” (acts like the request is older)
- **Instant Play Vote (\$3)**: +4 paid points but **cannot exceed cap**
- **Super Boost (\$5)**: sets a temporary flag: “if this song becomes #1 eligible, it gets picked next” (still must pass crowd approval)

That last bit is key: Super Boost doesn’t override hate.

---

## 7) Example (real numbers)

Song needs to “win” vs others.

- $up=12, down=3 \rightarrow net\_votes = 12 - 0.7*3 = 9.9$

- $\text{age}=18 \text{ min} \rightarrow \text{time\_boost} = \log(19)=2.94$
- $\text{paid\_points}=9 \text{ but cap}=6 \rightarrow \text{paid\_capped}=6$

Score  $\approx$

- $1.09.9 + 0.66 + 0.4*2.94$   
 $= 9.9 + 3.6 + 1.18$   
 $= \mathbf{14.68}$

If another song has up=18 but no paid, it may still beat it — which is the point.

---

## 8) UI that makes this feel fair

Show transparency without exposing the whole formula:

- “Crowd Support: 12 👍 / 3 👎”
- “Boost Power Used: 6/6 (max)”
- “Eligible: Yes (60%+ approval)”
- “Estimated position: #4”

People accept monetization when the rules are visible.

---

If you want, I can turn this into a **clean spec** for engineering:

- exact data model
- endpoints/events
- anti-abuse thresholds
- how to A/B test weights per venue type (dive bar vs club).
  - i.
  - d. Social/Engagement
    - i. Vibe Score

1. Real time reactions displayed on app (floating upon reaction and summary as seen further below) and maybe venue if display available (should we provide displays in theory?):
    - a. 👍 Fire
    - b. 💃 Danceable
    - c. 😊 Mid
    - d. 🙅 Skip
      - i. NOW PLAYING
      - ii. Drake – Passionfruit
- 
1. 🔥 62%
  2. 💃 25%
  3. 😊 8%
  4. 🙅 5%
- 
- ii. Leaderboard - Top DJs Tonight - rewards/discounts for leaders?
    - a. Alex – 9 songs played
    - b. Maya – 7 songs played
    - c. Sam – 6 songs played
      - i. Can be venue specific, geography specific, and time differentiated (tonight, weekly, all-time)
  2. Way to have people at different venues compete against each other?
    - a. Maybe for brand sponsored rewards?
  - iii. Pass the Aux
    1. Every [20] minutes the system selects a temporary crowd DJ who gets to queue up to [1] guaranteed songs to played next
      - a. Rule for who gets or random?
        - i. Random of people who've built greatest user influence scores for night
      - b. Builds user influence scores
      - c. Display: 🔥 Alex has the AUX for 10 minutes
    - iv. Announcements when you have the song up
    - e. AI Auto Queue
      - i. The algorithm decides songs so song continuity makes sense. For example, song scores based on:
        1. votes
        2. tempo
        3. genre continuity
        4. Key compatibility
        5. crowd reactions
        6. time of night
        7. venue preferences/venue audience/venue location

## B. Users

- a. Onboarding
  - i. Scan QR code at venue
  - ii. Auto-open web app (no download required initially)
  - iii. Apple/Google login OR phone number
  - iv. Auto-join venue session
- b. Live Venue Dashboard
  - i. Where music is good tonight? (think persistent leaderboards can lead to crowding out of smaller/lesser known venues)
    - 1. What metrics to use for this
      - a. Level of positive engagement (weight of pure activity vs positive scores)
  - ii. Where this genre has been popular tonight
  - iii. Try this tonight (in support of lesser known venues)
    - 1. Maybe increase positive effect on user influence scores/decrease negative while at this venue
    - 2. Some moderated, maybe some sponsored
- c. Social Profiles (not data side but minimal engagement between users)? Users can choose to have a 'public' profile or not
  - i. See genre tastes, most requested songs/artists
  - ii. Keep in mind, don't want to open up shaming based on influence scores (if we even decide to publicize those in some way)

### C. Venue

- a. Control Modes
  - i. Crowd Control - fully controlled by crowd but overridable at any time by venue
    - 1. Venues can block - lets consider this carefully, does it defeat whole point of supply house can just keep to its usual
      - a. explicit songs
      - b. genres
      - c. artists
      - d. tempo ranges, maybe after a certain time etc
  - ii. Suggestion mode - venue approves
- b. Dashboards
  - i. Crowd Mood Detection using signals like
    - 1. song skip rate
    - 2. vote velocity
    - 3. time of night
    - 4. genre engagement
    - 5. To show venue:
      - a. Crowd energy: HIGH
      - b. Trending: Afrobeat / hip hop
      - c. Low engagement: house